

# 10 个梯度实战案例（Agent 驱动开发任务书）

## 第一阶梯：基础控制与本地 I/O（培养指令语感）

本阶段目标：让学生习惯命令 Agent 操作系统环境、读取文件，体验最基础的“人机协作”。

### Case 1：进阶版 Hello World（CLI 参数解析器）

- 训练核心：**体验 Agent 自动安装依赖与处理终端参数的能力。
- 功能要求：**让 Agent 用 Python 或 Node.js 写一个命令行工具。运行 `python hello.py --name "张三" --lang "zh"` 输出中文问候，改用 `--lang "en"` 输出英文。
- 验收标准：**代码需包含完整的帮助文档（`--help`），并且 Agent 必须能自己在终端里运行测试并证明成功。

**踩坑提示：**告诉学生，如果 Agent 写错了，直接把终端报错复制给它，让它自己修，不要人工改代码。

### Case 2：Markdown 批量格式化大师

- 训练核心：**体验 Agent 的本地文件系统读写权限与批量处理能力。
- 功能要求：**在一个包含几十个混乱的 `.md` 文件的文件夹中，让 Agent 编写脚本：统一所有标题的层级（如把乱用的 `#` 和 `###` 规范化），并在每个文件头部自动插入 YAML frontmatter（包含处理时间和作者）。
- 验收标准：**脚本运行后，文件夹内所有文件格式整齐划一，且不能丢失原文内容。

**踩坑提示（重要）：**Agent 很容易在批量修改时把文件写空！引导学生在 Prompt 中加上：“在修改前，先自动创建一个 backup 文件夹备份所有文件。”——培养系统的容错防御意识（Harness 控

制)。

### Case 3: 基于 JSON 的持久化 Todo List

- **训练核心：** 数据持久化与状态管理 (CRUD) 。
- **功能要求：** 一个终端版的待办事项工具，支持 Add、List、Done、Delete 四种命令。所有数据必须持久化保存在本地的 `data.json` 文件中。
- **验收标准：** 关闭终端再重新打开，输入 List 依然能看到之前添加的任务。JSON 数据结构需清晰 (包含 id、title、status、timestamp) 。

**踩坑提示：** 当功能增加时，Agent 可能会忘记 JSON 的字段格式。教学生使用"契约": "请先向我输出你设计的 JSON 数据结构，我确认后再开始写代码。"

## 第二阶梯：跨域交互与架构拆分（对抗上下文腐化）

本阶段目标：打破单文件开发的舒适区，引导学生面对复杂系统的模块化拆分，以及不可见的 UI 调试。

### Case 4: 极简网页数据爬虫

- **训练核心：** 网络请求处理与第三方 DOM 结构解析。
- **功能要求：** 指挥 Agent 抓取某个静态公开网站（如 Hacker News 或豆瓣 Top250 静态页）的标题、链接和评分，并将数据清洗后导出为 `result.csv` 。
- **验收标准：** 成功抓取至少 20 条数据，CSV 格式正确不乱码。

**踩坑提示：** AI 极容易"幻觉"网页的 HTML 结构（凭空捏造 Class name）。教学生：先让 Agent 写一个脚本把网页 HTML 源码拉取下来并打印部分结构，让 Agent 自己看一眼源码，再让它去写最终的解析逻辑。

---

## Case 5: 桌面 GUI 专注时钟 (PyQt / Tkinter)

- **训练核心:** 体验充当 AI 的"眼睛", 建立跨模态的反馈闭环。
- **功能要求:** 一个带有界面的番茄钟。包含倒计时显示、开始/暂停按钮。
- **验收标准:** 界面可以正常启动, 按钮点击有响应, 倒计时准确。

**踩坑提示:** 纯文本 Agent 看不到弹出的 UI 界面! 如果按钮太小或颜色很丑, Agent 是不知道的。学生必须学会充当视觉反馈器: "现在的按钮全挤在左上角, 请帮我用 Grid 布局居中, 并把字体放大到 24px。"

---

## Case 6: 多模块 RESTful API 后端

- **训练核心:** 对抗"上下文腐化", 强制 AI 进行工程化拆分。
- **功能要求:** 开发一个图书管理的后台 API (增删改查)。必须使用框架 (如 Express 或 FastAPI)。绝对禁止写在一个文件里。
- **验收标准:** 代码必须严格拆分为 `app.js` (入口)、`routes/` (路由)、`controllers/` (控制器) 多个文件。通过 curl 或 Postman 测试 API 连通性。

**踩坑提示:** 如果只说"帮我写个 API", Agent 为了省事会把所有逻辑全塞进一个几百行的代码里, 随后必然发生逻辑混乱。教学生: "先设计目录树给我看, 然后依次实现每一个文件。"

---

## 第三阶梯: 系统级协作与自我进化 (Harness 与 Meta 实战)

本阶段目标: 高难度综合挑战。模拟最真实的软件开发困境, 逼迫学生使用"剪枝"、"回滚"、"环境隔离"等高阶 Agent 驾驭术。

## Case 7: 前后端打通的全栈博客

- **训练核心:** 跨技术栈的综合上下文管理。
- **功能要求:** 结合 Case 6 的后端, 让 Agent 再写一个前端页面 (HTML + JS)。前端通过 fetch 请求后端 API, 把图书列表渲染到网页上。
- **验收标准:** 前端页面能在浏览器打开, 且能成功展示来自后端 API 的动态数据 (解决跨域 CORS 问题)。

**踩坑提示:** 此时对话轮数极多, Agent 大概率会"失忆"或混淆前后端的端口号。教学生: 要求 Agent 编写并维护一个 `ARCHITECTURE.md`, 每次换模型或重启对话时, 先让 Agent 读这个文档恢复记忆。

## Case 8: 外部第三方 API 整合看板 (天气 + 图表可视化)

- **训练核心:** 陌生知识注入与 API Key 的安全隔离。
- **功能要求:** 调用真实的外部 API (如心知天气或 OpenWeather), 获取数据后, 使用 Chart.js 或 ECharts 在网页上画出未来三天的温度折线图。
- **验收标准:** 图表正确渲染。代码中绝对不能硬编码明文的 API Key。

**踩坑提示:**

- Agent 如果不知道最新的 API 格式, 让学生去官网把 API 文档 (JSON 示例) 复制喂给 Agent。
- 强迫 Agent 学习使用 `.env` 文件读取环境变量。

## Case 9: 屎山代码 (Shit-Mountain) 重构与逆向 Debug

- **训练核心:** 锻炼 Harness 强制干预能力, 打破死循环。
- **功能要求:** (老师需提前准备一段故意写得很烂、有语法错误、嵌套 5 层循环的迷宫代码)。要求学生用 Agent 找出 Bug 并重构为面向对象的优雅代码。
- **验收标准:** 重构后的代码能跑通原来的逻辑, 且圈复杂度大幅降低。

**踩坑提示：** Agent 在面对屎山时，极容易陷入"修改 → 报错 → 继续错误修改"的死循环。学生必须学会"按 Ctrl+C 强行打断 Agent 的施法"，然后严厉地对 Agent 说："停止盲目修改！先一步步向我解释这行代码为什么会报错，提出方案我同意后你再动代码。"

### Case 10：终极奥义 —— AI 召唤 AI (Meta-Programming)

- **训练核心：** 理解 AI 时代的最强范式（套娃），为第四阶段课题打底。
- **功能要求：** 引导 Claude Code Agent 编写一个 Python 脚本。这个 Python 脚本的功能是：读取指定文件夹里的数十段长文本（txt），然后在代码里调用 DeepSeek 或 Kimi 的大模型 API，自动总结每一篇文本的摘要，并重命名该文件。
- **验收标准：** 脚本跑通。实现"你指挥 Agent A，Agent A 写出代码，代码自动调用 Agent B 批量干活"的奇观。

**踩坑提示：** Agent 可能会把构建脚本本身的逻辑，和脚本运行后去调用别的 API 的逻辑搞混。需要极其清晰的逻辑隔离指导："你现在的任务是帮我写代码，代码里用到的是另一个 API Key，不要在写代码的时候就去发请求。"

### 评估与打分体系指南

在这个阶段的末尾，不对代码的"优雅程度"做过高要求（毕竟是 AI 写的），核心评估指标如下：

| 评估维度                 | 说明                                                                    |
|----------------------|-----------------------------------------------------------------------|
| AI 幻觉降维率             | 记录在开发过程中，Agent 发生报错的次数。报错越少，说明该学生下达的 Prompt 越清晰。                      |
| 工程纪律性                | 检查 <code>.env</code> 文件是否被正确使用，API Key 是否有泄露？是否养成了让 AI 先备份再操作的习惯？     |
| 干预决策力<br>(Harness 力) | 在 AI 陷入死循环时，学生是眼睁睁看着它烧掉几十万 Token，还是能果断止损、回滚上下文、强行重定向逻辑？（可作为复盘会上的分享考核） |